

gif-server — Deployment Guide

Self-hosted deployment to a Docker homeserver behind a containerized nginx reverse proxy. Updated 2026-07-11.

1. Architecture

Three pieces, all on the homeserver:

- **db** — Postgres 16 container (gif-server's compose stack).
- **app** — the gif-server binary container. Migrations run automatically on startup. Bound to `127.0.0.1:<APP_HOST_PORT>` on the host (for debugging) and joined to the existing nginx Docker network with alias **gif-server**.
- **nginx** — a pre-existing container (`lscr.io/linuxserver/nginx`) on the nginx network. The sole public ingress; terminates TLS and proxies to `http://gif-server:8847`.

The app is never published to the internet. The internet reaches it only through nginx.

2. Prerequisites

- Docker + Docker Compose on the homeserver (already present).
- The existing nginx container *and the Docker network it is attached to*. Note the network's real name (see the box below) — it is often *not* literally **nginx**.
- DNS A/AAAA record for the public domain (e.g. `gifs.example.com`) pointing at the homeserver's public IP.
- Router port-forward of TCP **80** and **443** to the homeserver.
- A TLS certificate for the domain, mounted into the nginx container (handled separately).
- A few GB of free RAM/disk — the first Rust build is slow and memory-hungry.

*The **nginx** network name. `docker-compose.yml` joins an **external** network. Compose does a literal lookup, so the network must exist by the exact name given. Having an **nginx** container is not the same as having a network named **nginx**: a container started by its own compose project is usually on a project-prefixed network like **nginx_default**. Find the real name with `docker network ls` (or `docker inspect <nginx-container>`), then set it in the **name:** field of the network block — see Section 7.*

3. Repository files involved

docker-compose.yml

db + app stack. Config comes from `.env` via `${VAR}` interpolation. App joins the external nginx network.

.env.example

Template for configuration. The real `.env` is created on the server and is git-ignored / not rsynced.

deploy/nginx.conf.example

Reverse-proxy server block to copy into the nginx container's site-confs.

deploy/backup.sh

Backs up the GIF files + database to a timestamped folder, then verifies both archives. See Section 11.

deploy/restore.sh

Restores a backup produced by `backup.sh`. See Section 11.

`deploy/deployment-guide.tex`

Source for this PDF. Edit it and recompile rather than editing the PDF directly.

Dockerfile

Multi-stage Rust build (cargo-chef) producing a slim runtime image.

4. Step 1 — Copy the code to the server

From the development machine, in the project directory:

```
rsync -avz --delete \
  --exclude target/ --exclude .git/ --exclude /data/ --exclude .env \
  ./ user@homeserver:/srv/gif-server/
```

Excludes the build cache, git history, the local dev data dir, and the env file. Run once with `-n` (dry run) first to preview, then without.

Production GIF storage lives outside the deploy dir — it is a bind mount to the host path in `STORAGE_HOST_PATH` (set it to `/srv/gif-data`, a sibling of `/srv/gif-server`). Because that path is not under the `rsync` target, a `-delete` deploy cannot reach it regardless of excludes; because it is a bind mount and not a Docker volume, `docker compose down -v`, `docker volume prune`, and `docker volume rm` cannot reach it either. Only an explicit `rm` on `/srv/gif-data` deletes it. The `-exclude /data/` above is now only hygiene, skipping the git-ignored local dev data dir so it is not pushed. Still anchor directory excludes with a leading slash (`/data/`, not `data/`): an earlier unanchored `-exclude storage/` silently dropped the `src/storage/` module and broke the build (`error[E0583]: file not found for module storage`).

First time: ensure the target exists and is writable by your SSH user:

```
sudo mkdir -p /srv/gif-server
sudo chown "$USER:$USER" /srv/gif-server
```

5. Step 2 — Create the production `.env` on the server

At `/srv/gif-server/.env` (it is not rsynced, so create it fresh). Generate secrets with `openssl rand`:

```
POSTGRES_PASSWORD=<openssl rand -hex 24>
SESSION_SECRET=<openssl rand -hex 32>
BASE_URL=https://gifs.example.com
MATRIX_SERVER_NAME=matrix.example.com
MATRIX_FEDERATION_URL=https://matrix.example.com
MATRIX_ADMIN_MXIDS=@admin:matrix.example.com
CORS_ALLOWED_ORIGINS=https://<your-frontend-origin>
# REQUIRED in production: put the GIF files OUTSIDE the deploy dir.
STORAGE_HOST_PATH=/srv/gif-data
# Optional: override host-published ports only if they clash with other
# host services (the app/nginx use the internal network, not these).
DB_HOST_PORT=5433
APP_HOST_PORT=8848
```

- `STORAGE_HOST_PATH` is what puts the GIF files at `/srv/gif-data`, a *sibling* of the deploy dir. **Do not omit it.** Compose falls back to `./data`, which resolves *inside*

/srv/gif-server — the exact thing the safety note in Section 4 says the production layout avoids.

- DATABASE_URL is **not** needed here — compose builds it from POSTGRES_PASSWORD with host db.
- SESSION_SECRET must be at least 32 characters.
- BASE_URL must be the public https:// URL — it is embedded in rendition URLs.
- MATRIX_FEDERATION_URL is where the homeserver’s federation API is reachable. If the server delegates federation to port 443 via .well-known/matrix/server, this is just https://<server-name> (no :8448).
- DB_HOST_PORT / APP_HOST_PORT set the host ports the db and app containers publish on 127.0.0.1 (defaults 5432 / 8847). Nothing in the running system depends on them — the app reaches Postgres at db:5432 and nginx reaches the app at gif-server:8847, both over the internal network — so override them only to dodge a host-side port clash. If you change APP_HOST_PORT, adjust the host-side curl .../health checks accordingly.
- STORAGE_MAX_BYTES / PER_USER_STORAGE_BYTES fall back to defaults (10GB / 1GB) if omitted.

Create .env before the first docker compose up. Required variables use \${VAR:?} markers, so compose fails loudly (by design) if any are missing.

6. Step 3 — Build and start the stack

```
cd /srv/gif-server
docker compose up -d --build
docker compose logs -f app          # watch migrations apply + startup
curl -fsS http://127.0.0.1:8847/health # expect HTTP 200 from the host
```

A clean boot logs the migrations (001_init.sql, 002_hidden.sql) then “listening on 0.0.0.0:8847”.

7. Step 4 — Wire up the nginx network and site

Network. If docker compose up fails with network nginx declared as external, but could not be found, point the compose network block at the real network name reported by docker network ls:

```
# docker-compose.yml
networks:
  default:
    nginx:
      external: true
      name: nginx_default      # <-- the actual network the nginx container is on
```

Alternatively, create a network literally named nginx and attach the container:

```
docker network create nginx
docker network connect nginx <nginx-container>
```

Site. Copy deploy/nginx.conf.example into the nginx container’s site-confs directory on the host:

```
cp /srv/gif-server/deploy/nginx.conf.example \
  /docker/apps/nginx/nginx/site-confs/gif-server.conf
```

Edit it: set the real `server_name` and the container-internal TLS cert paths. Key points already baked into the template:

- Proxies to `http://gif-server:8847` using Docker's embedded DNS resolver (127.0.0.11) and a variable upstream, so an app restart doesn't strand nginx on a stale IP.
- `client_max_body_size 50m`; — matches the app's 50 MB upload limit (nginx defaults to 1 MB).
- Overwrites `X-Forwarded-For` / `X-Real-IP` with `$remote_addr`.

Then validate and reload nginx:

```
docker exec <nginx-container> nginx -t
docker exec <nginx-container> nginx -s reload
curl -fsS https://gifs.example.com/health # expect HTTP 200 over TLS
```

8. Step 5 — Verify

- **App resolvable from nginx:** `docker exec <nginx-container> getent hosts gif-server` returns an IP.
- **Real client IPs:** nginx access logs show public client IPs, not `172.x` gateway addresses — confirms the per-IP rate limiter keys on real clients.
- **Auth path:** `POST /auth/matrix` with a real Matrix OpenID token returns a JWT; bad tokens return 401 (not 500).

***Security — X-Forwarded-For.** The auth rate limiter trusts the leftmost `X-Forwarded-For` value. nginx must **overwrite** it with `$remote_addr` (the template does), **not** append via `$proxy_add_x_forwarded_for` — appending would let a client prepend a spoofed IP and bypass the per-IP login limiter. The app must remain reachable only through nginx (it is: `localhost` bind + internal network, never publicly published). A non-spoofable global auth cap (~5 req/s) bounds total login load as a second layer.*

9. Local development (unchanged)

Local dev runs the app on the host against the dockerized db:

```
./scripts/dev.sh # starts db via compose, waits healthy, runs cargo run
# or, if db is already up:
cargo run
curl -i http://localhost:8847/health
```

- The app reads the local `.env` (localhost DB URL, localhost base URL).
- `docker compose up -d db` works locally despite the external nginx network (only the app service references it).
- Running the **full** stack locally (`docker compose up`) requires the network to exist: `docker network create nginx` first.

10. Routine operations

Redeploy after code change

`rsync` (Step 1), then `docker compose up -d -build`. The `-build` is required: the binary is compiled *into* the image, so `docker compose restart` re-runs the *old* image and will not pick up new code.

View app logs

```
docker compose logs -f app
```

Restart app only

`docker compose restart app` — restarts the current image; use it to clear a wedged process, *not* to deploy code.

Stop everything

`docker compose down` — GIF files are unaffected (they are a host bind mount, not a Docker volume); the `postgres\_data` volume also persists. `down -v` deletes `postgres\_data` but still cannot touch the bind-mounted GIF files.

11. Backup and restore

GIF files live on the host at `STORAGE_HOST_PATH` (`/srv/gif-data` in production), root-owned (the container runs as root). The database lives in the `postgres\_data` Docker volume. The two are coupled: rows reference GIF files by hash, so they must be captured and restored **together**.

Back up (writes a new timestamped folder under `/srv/gif-backups`; deletes nothing):

```
cd /srv/gif-server
./deploy/backup.sh
```

It writes `gif-storage.tgz` + `gifserver-db.sql.gz`, then verifies both: it compares the archive's file list against `STORAGE_HOST_PATH` and checks the dump for the `pg_dump` completion marker, failing loudly if the dump is truncated. Backups are additive — prune old ones yourself.

Restore (destructive to current state — see the box below):

```
cd /srv/gif-server
docker compose stop app      # restore.sh refuses to run while the app is
                             # connected
./deploy/restore.sh latest   # or a timestamp, or an explicit path
docker compose up -d app
```

`restore.sh` verifies the archives *before* touching anything, so a corrupt backup aborts without disturbing live data. It then moves the current GIF dir aside to `<storage>.pre-restore-<stamp>` (it does not delete it), extracts the backup, drops and recreates the database, and replays the dump. Delete the preserved directory yourself once you have verified the result.

Restore both halves from the same folder. *Mixing a dump from one run with files from another leaves rows pointing at GIF files that do not exist. The database is dropped and recreated* — data written since the backup is lost. Neither script starts, stops, or recreates a container; both run `pg_dump/psql` inside the already-running `db` container via `docker compose exec`, and both refuse to run if `db` is down. **Copy backups off the box** — `/srv/gif-backups`

is out of reach of an `rsync -delete` deploy (it is not under `/srv/gif-server`), but it is on the same disk and does not survive hardware failure.

12. Troubleshooting

error[E0583]: file not found for module storage (plus a cascade of E0282)

The `src/storage/` module never reached the server. Cause: an unanchored `rsync` exclude (`storage/`) matching the source dir. Fix: use `-exclude /data/` (the data dir is named `data`, so it no longer collides), and confirm `src/storage/` is present on the server before building.

Bind for 0.0.0.0:5432 failed: port is already allocated

Another Postgres on the host already owns the published port. Set `DB_HOST_PORT` in the server's `.env` to a free port (e.g. 5433). The app is unaffected — it uses the internal `db:5432`, not the host port.

network nginx declared as external, but could not be found

No Docker network with that exact name exists. Set the `name:` field of the compose network block to the real network (Section 7), or create/attach a network literally named `nginx`.